Type inference in LAMBDA

Constraints, annotations, and explicit polymorphism

CS413 / LAMBDA

From typing constraints to the implementation in `lambda2.py`

Inference as constraint solving

Start with unknown types for expressions whose types are not yet known.
Each language construct adds equations between types. Substitution solves the equations. A conflict means the program is ill typed.

The resolved type of the whole expression is the checker result.

```
lam (x) => x + 1

parameter: ?t0
+ requires: ?t0 = int
result: int
function: int -> int
```

Omitted types are inferred. Inferred bindings are not generalized.

Types and inference variables

$$T ::= \texttt{int} \mid \texttt{bool} \mid \texttt{unit} \mid (T_1, \ldots, T_n) \mid T_1 \to T_2 \mid \texttt{List}[T] \mid \texttt{Optn}[T] \mid ?t_N$$

A type may contain unknowns, for example $?t_0 \to \texttt{List}[?t_1]$.

- `solver.fresh()` creates an unknown `?tN`.
- A substitution records what an unknown stands for.
- `solver.apply(T)` follows substitutions through a type.

Concrete types: `TInt`, `TBool`, `TTuple`, `TFun`, `TList`, `TOption`, and `TMeta`. Declared type variables are represented as `TPoly` before checking.

Unification turns uses into type facts

Two function types unify by matching their arguments and results. Tuple types unify component by component when their lengths match. Lists and options unify their element types. A metavariable can be replaced when that does not create an infinite type.

```
unify(?t0, int)
  => ?t0 := int
```

```
unify(?t1 -> bool, int -> ?t2)
  ?t1 := int
  ?t2 := bool
```

A complete inference example

```
lam (x) => x + 1
```

1. Give `x` a fresh type, `?t0`.
2. The `+` rule requires both operands to have type `int`, so `?t0 = int`.
3. The literal `1` has type `int`; the body therefore has type `int`.
4. The lambda type `?t0 -> int` resolves to `int -> int`.

The typing environment

`d0exp_typecheck` checks each syntax tree node. Its environment maps names to type schemes.

- A monomorphic binding has no quantified variables.
- An explicitly polymorphic function quantifies only its declared type parameters.
- A lambda extends a copied environment with its parameter type.

```
inner = dict(env)
inner[param] =
  monomorphic(parameter_type)
```

A recursive function binds both its function type and its parameter in the body. When a declaration gives a function signature, the checker uses it as context to infer omitted lambda and recursive parameter types.

Constraints by construct

Conditional

```
if c then a else b
  unify(type(c), bool)
  unify(type(a), type(b))
```

Application

```
f(arg)
  unify(type(f), type(arg) -> fresh())
```

Arithmetic

```
x + y
  unify(type(x), int)
  unify(type(y), int)
```

Container primitives constrain element types. The checker reports `TypeCheckError` at a conflicting constraint.

Explicit polymorphism, without generalization

```
identity[T] : T -> T = lam (x) => x
```

- `T` is declared in the function name's type parameter list.
- The function needs an explicit signature. The lambda parameter type is inferred from it.
- Each use instantiates `T` with a fresh type.

```
identity(1)      # T is int at this use
identity(true)   # T is bool at this use
```

The same syntax works for inner functions in local declaration blocks. Functions without explicit type parameters remain monomorphic. Inferred types are never generalized.

Annotations constrain inferred types

```
(expression : T)
```

- `D0Eanno` wraps an expression and its declared type.
- The checker infers the expression type, then unifies it with `T`.
- Declaration signatures also guide inference inside function bodies.

```
(lam (x) => x + 1 : int -> int)
# accepted

(1 : bool)
# rejected

id[T] : T -> T = lam (x) => x
# accepted
```

What the implementation checks

- Applications, arithmetic, comparisons, conditionals, tuples, and projections.
- Homogeneous list and option operations, with element types inferred from use.
- Recursive function bodies against their result types.
- Expression and declaration annotations, including explicit polymorphic signatures.
- Unbound names, tuple index errors, mismatched types, and infinite types.

```
d0exp_parse_and_typecheck(source)
  expression = d0exp_parse(source)
  return expression, d0exp_typecheck(expression)

d0cls_typecheck(d0cls_parse(source))
```